Memory that survives the router:

a head-to-head study of cross-model recall and verifiable provenance for LLM agents behind a model gateway

SenseLab

https://amfs.sense-lab.ai

July 2026

Abstract

Model routers such as OpenRouter send successive calls from one agent to different model providers to optimize cost and availability. Because an LLM API call is stateless, a fact stated under one model is absent for the next, and current models do not abstain when a fact is missing—they fabricate. We quantify this and test six conditions on a fixed dataset and grader (144 evaluations per condition across three cross-vendor model pairs). With no memory layer, a routed agent answers a follow-up question correctly only **12.5%** of the time [8.1%, 18.9%]; fabrications are confident, specific, and raise no error. A memory layer restores recall to the in-context ceiling, but this is true of *every* competent layer we tested: a local vector store (100%), mem0 (99.3%), and SenseLab (100%) are statistically indistinguishable, with Supermemory lower (86.1%) largely because of asynchronous ingestion. Recall, in other words, is table stakes. The dimension that separates the systems is what each can *prove* about an answer afterward. For all 144 answers, SenseLab emitted a signed, hash-chained decision trace that verified against tampering (a modified trace failed verification on both content hash and signature), linked each answer to the specific versioned facts and the model that produced it, and passed write-time safety checks; the other three systems emit no such artifact. We also report an honest negative result: abstention on unanswerable questions is a tunable precision/recall tradeoff, not a free property, and at the recall-maximizing setting SenseLab is mid-pack. All code, raw JSON, and figures are released for reproduction.

1 Introduction

We took one agent, told it an operational fact through OpenRouter, then asked a follow-up after the router had moved the agent to a different vendor's model. Nothing but the model changed. With no memory layer the agent answered correctly **12.5%** of the time (18 of 144 runs, 95% Wilson CI [8.1%, 18.9%]).

The 87.5% that were wrong is the part worth dwelling on. The failures were not errors or refusals. They were confident, specific, invented answers. Asked for a database version it had been told was PostgreSQL 15, one run replied `5.7.31-0ubuntu0.18.04.1`—a real-looking MySQL string—and named the wrong deploy day, with no hedge. Nothing in the response signals a guess, and nothing appears in a log as an error. It looks like an answer.

Two controls make the cause unambiguous. Handing the same models the fact inside the prompt yields 98.6% correct: the models are capable, they simply lack the fact. Putting a competent memory layer in front of the router restores recall to that ceiling. Most memory products stop there. This paper starts there, because on recall the memory layers are interchangeable; the difference that survives is what each can prove about the answer once it has been produced.

Every claim below has a checkable form. Code, grader, and raw JSON are released (Appendix A) so any step can be re-run and disputed.

2 The mechanism of the failure

"Agents forget" is too vague to act on. The failure decomposes into four steps, each verifiable:

1. An LLM API call is stateless. The model retains nothing between calls; continuity comes only from what is resent in the request.
2. A router sends call $N{+}1$ to a different provider than call N, chosen on price or availability. This is the purpose of a gateway, not a misconfiguration.
3. Nothing carries prior state across that hop. Two separate requests share no context window, and provider-native memory is scoped to one provider, so it does not follow.
4. When the needed fact is absent, current models do not abstain. They complete the pattern with a fluent, plausible value. Absence becomes fabrication rather than "I don't know."

Steps 1–3 are properties of the stack, confirmable from the API contracts. Step 4 is empirical and is measured in Section 4. The danger is that a fabricated answer is indistinguishable from a correct one without the ground truth and raises no exception, so it is invisible to ordinary monitoring and surfaces downstream—as a wrong action or a customer complaint—detached from its cause.

Scope of the claim. This applies if one agent's traffic is routed across multiple providers and the agent relies on facts established earlier. It does not apply to a single model from a single vendor inside one context window, or to an agent that is stateless by design; there a memory layer only adds moving parts.

3 Method

The honest baseline is not "does memory beat no memory"—it does—but "does this beat the memory layers people already run, and does it give something they cannot?" We ran six conditions against the same dataset and grader.

Dataset. Eight durable operational facts (production database version and deploy day; on-call engineer and extension; region; internal billing API URL; SEV-1 acknowledgement window; key-rotation cadence; top customer and renewal date; merge-approval rule). Each has a question and keyword-based grading.

Model pairs (writer → reader, always crossing vendors):

- `openai/gpt-4o-mini` → `mistralai/mistral-nemo`
- `mistralai/mistral-nemo` → `amazon/nova-micro-v1`
- `amazon/nova-micro-v1` → `cohere/command-r7b`

Trials. Six per (fact × pair): **144 evaluations per condition**, 48 per pair. Abstention: eight questions whose answers were never stored, six trials each (48 per condition).

Conditions.

- **no-memory** — reader answers with no prior context (fabrication floor).
- **in-context** — the fact is pasted into the reader's prompt (capability ceiling; isolates memory transfer as the only variable).

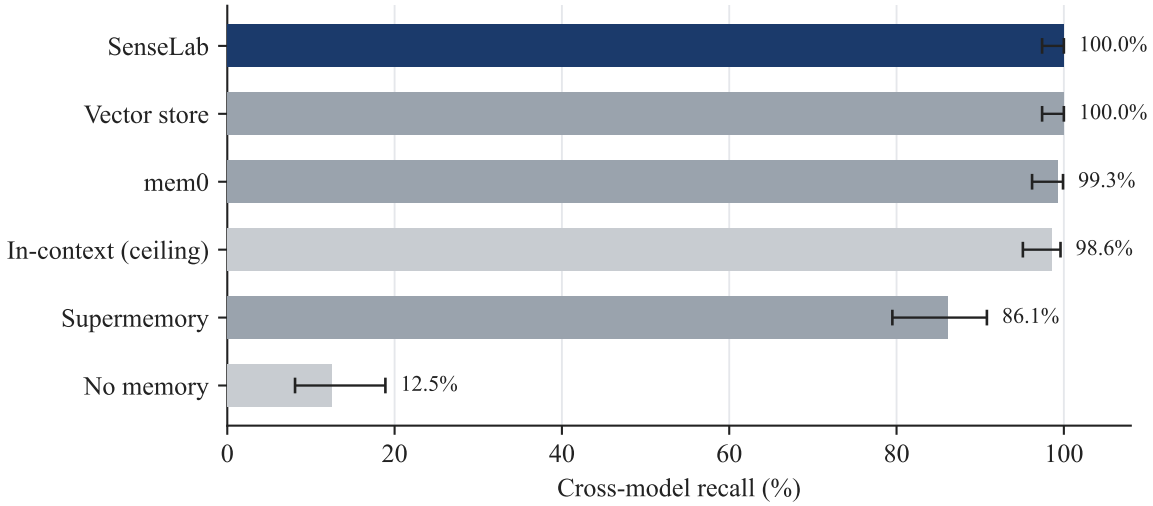

Figure 1: Cross-model recall by condition ($n = 144$ each), with 95% Wilson confidence intervals. No-memory and in-context are controls (grey); SenseLab is highlighted. The four memory layers cluster near the in-context ceiling; the gap to no-memory (12.5%) is the effect under study.

- **vector** — a local embedding store (`fastembed`, bge-small) with top-k cosine retrieval.
- **mem0** — the hosted mem0 platform, seeded and queried through its live API.
- **supermemory** — the hosted Supermemory platform, live API.
- **SenseLab** — multi-strategy retrieval (semantic + BM25 + confidence) with a confidence and semantic-relevance gate, write-time safety validation, and a signed, sealed decision trace per answer.

Grading. Keyword match against the known answer, identical across conditions. Wilson score intervals for all proportions. All numbers are from one live run against the hosted services.

4 Results

4.1 Recall across the model switch

Every serious memory layer clears the bar, and SenseLab sits at the top of a tight pack rather than alone (Figure 1). Vector and SenseLab reach 100%, mem0 99.3%; their intervals overlap. Supermemory landed lower (86.1%), but read that carefully: its ingestion is asynchronous, and in several scopes a just-written fact was not yet searchable when the question fired—an operational property, not evidence of weak retrieval. The defensible conclusion is that recall is table stakes and does not, by itself, separate the systems.

Per model pair ($n = 48$ each), failures do not cluster in one pair:

4.2 Overhead

Injecting a retrieved fact adds ∼36 prompt tokens on these short prompts. Model-side latency dominates and swamps every layer's own cost, so the memory arms are not slower than no-memory (Table 2). We claim no latency win; the point is that retrieval overhead is not a reason to prefer one layer.

Pair (writer → reader)	no-memory	SenseLab
gpt-4o-mini → mistral-nemo	8.3%	100%
mistral-nemo → nova-micro	4.2%	100%
nova-micro → command-r7b	25.0%	100%

Table 1: Per-pair recall. The no-memory rate varies with the reader model, but the memory arm restores full recall across all three pairs.

Condition	p50 latency	p95 latency	Mean prompt tokens
no-memory	840 ms	2284 ms	16.7
in-context	614 ms	1501 ms	35.7
vector	580 ms	1713 ms	51.9
mem0	603 ms	1637 ms	56.9
Supermemory	624 ms	2170 ms	55.9
SenseLab	563 ms	1644 ms	52.3

Table 2: Reader latency (p50/p95) and mean prompt tokens per condition.

4.3 Abstention on a miss (an honest negative result)

We asked eight questions whose answers were never stored and measured how often each arm declined instead of inventing an answer (Figure 2). At the gate setting that maximizes recall, **SenseLab does not win abstention**—it sits mid-pack (41.7%), behind mem0 (56.2%) and the vector store (47.9%). Abstention is not a property one obtains simply by choosing SenseLab; it is a knob, and the next subsection shows the knob's cost.

4.4 The gate tradeoff

SenseLab's semantic-relevance floor (SEM_FLOOR) decides whether a retrieved entry is relevant enough to inject. Because that decision is purely a function of the embeddings, it can be swept exactly, with no LLM calls (Figure 3). The recall and abstention curves move against each other, and the classes overlap: the hardest true question–fact pair has similarity 0.592, while the worst miss question's nearest stored fact scores 0.686. **No single global threshold cleanly separates a real question from a miss** on this embedder. One can tune SenseLab to abstain on 100% of misses (floor 0.70) at the cost of dropping recall to 87.5%, or tune for full recall and accept that a quarter of misses leak through. A per-entry or learned threshold, or a stronger embedder, would separate the classes better; that is future work, not a shipped claim.

5 Provenance and governance: the differentiator

Recall parity is where the memory market lives. The property that is hard to obtain elsewhere is a per-answer record that ties the output to specific, versioned, hashed facts and to the model that produced it, and that can be cryptographically verified and shown to detect tampering. For every one of the 144 SenseLab answers, the proxy sealed a record of this form (real, from the run):


```
{
  "outcome_ref": "req-0a83a424a0",
  "routed_model": "mistralai/mistral-nemo",
  "cost_usd": 1.29e-06, "prompt_tokens": 55, "completion_tokens": 10,
```

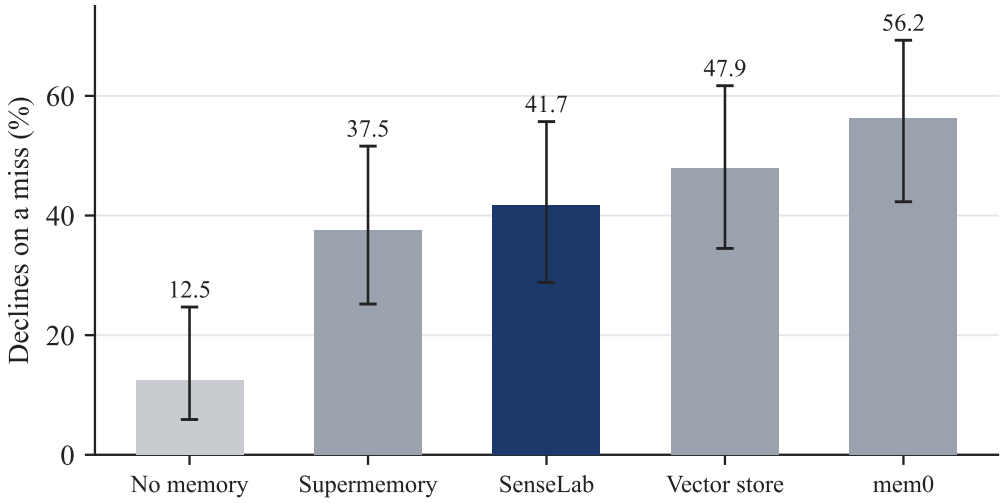

Figure 2: Fraction of unanswerable questions each system declined rather than fabricated ($n = 48$ each), with 95% CIs. At the recall-maximizing gate (0.55), SenseLab is mid-pack.


```
  "memory_used": [{"key": ".../fact-fb367271", "semantic": 0.818, "confidence": 0.9}],
  "content_hash": "45ce6cc19207a371...",
  "signature": "109bd0917fdc7fce...", "signing_key_id": "hmac-v1",
  "parent_hash": null, "sequence_number": 0,
  "trace_verified": true,
  "answer_excerpt": "PostgreSQL 15, Fridays"
}
```


What we *measured* on this artifact, rather than asserted:

- **Coverage.** A signed, verified trace was produced for 144 of 144 answers (100%), each naming the model that produced it.
- **Tamper-evidence.** We altered a sealed trace and re-verified; verification failed with a `content_hash` mismatch and a signature-verification failure. The clean trace verified; the altered one did not.
- **Chain integrity.** Sequential traces link by `parent_hash` / `sequence_number` into a Merkle-style chain that verified end-to-end.
- **Write-time safety.** A validator flagged a write that contradicted an existing high-confidence entry, and flagged a low-confidence write, before either was committed.

The claim any reader can check against the vendors' own APIs: a vector store, mem0, and Supermemory return retrieved text and a completion. **None emit a signed, content-hashed, model-attributed, verifiable causal trace.** In this run, the count of tamper-evident decision traces produced by the four memory arms was 144 for SenseLab and 0 for the others (Table 3). That is a structural property of what each system stores, not a tuning gap, and it is the only dimension on which the arms were not close.

6 Practitioner implications

The same result reads differently depending on what you are accountable for.

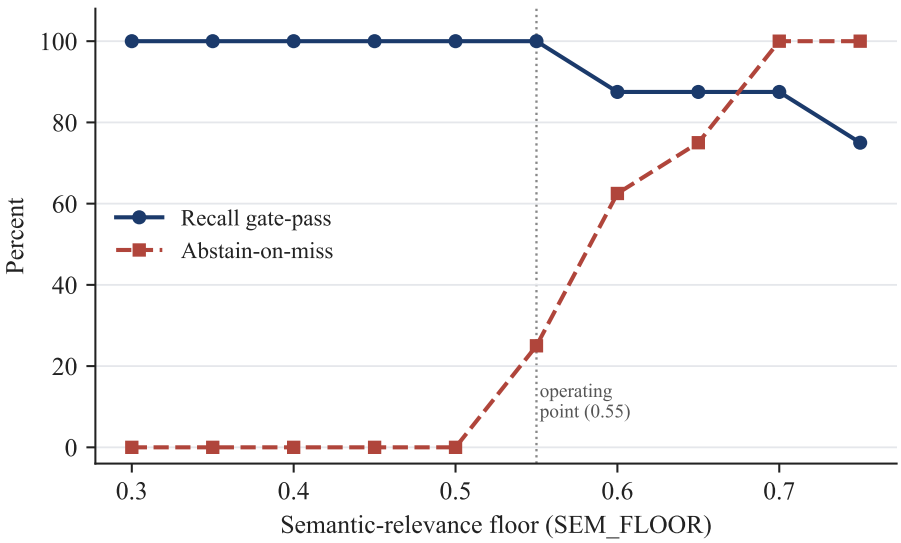

Figure 3: Deterministic gate sweep. Raising the relevance floor increases abstention on misses but lowers recall gate-pass. The dotted line marks the operating point (0.55) used for the end-to-end run.

Capability	vector	mem0	Supermemory	SenseLab
Restores cross-model recall	yes	yes	partial	yes
Signed per-answer trace	no	no	no	144/144
Tamper detected on modification	n/a	n/a	n/a	yes
Verifiable hash chain across a session	no	no	no	yes
Write-time contradiction check	no	no	no	yes
Tunable abstain gate (measured)	no	no	no	yes

Table 3: Governance capabilities observed in the run. Recall is shared; verifiable provenance and write-time validation are not.

For engineers. Be suspicious of anyone selling recall—on recall a 50-line vector store matches the hosted platforms. The reason to reach for a governed layer shows up the day something breaks: when the next "why did it say that" lands, the sealed trace gives you the exact fact read, its confidence, the model that answered, and the cost, as a lookup rather than an archaeology project, with a signature proving the record was not edited. Adoption is a `base_url` change plus two headers; the layer fails open, so if it errors the request still reaches the router.

For engineering managers. Cost optimization has a failure mode: you optimize the number you can see (the inference bill) and push cost into the ones you cannot (support tickets, re-asked questions, wrong actions), none tagged "caused by routing." The memory layers are within one latency band (Table 2), so this is not a performance tradeoff. The differentiator worth paying for is the per-decision ledger: which model produced each answer, on what memory, at what cost—verified for 144 of 144 answers. That is the data routing was supposed to give you and quietly took away.

For risk and compliance. A prompt and completion in a log is not provenance: it cannot show which of several models answered, what it relied on, or whether that was real. A governed memory layer captures verifiable provenance at decision time and can decline rather than guess. Be precise

about scope: hashing proves a stored value was not altered after the fact; it does not prove the value was true when written. Write-side truth remains a control you own; the validator narrows the gap, it does not close it. The signing key here is a development HMAC key—production sets a managed key, same mechanism.

7 Limitations

- **Abstention is a tradeoff, not a win.** At the recall-parity gate, SenseLab abstains on 41.7% of misses, behind mem0. Section 4 is the honest picture.
- **One dataset, one grader, one run.** Eight facts, keyword grading, a single live run. The recall effect is large and the CIs are reported, but a second independent run and an adversarially-worded dataset would strengthen it. The governance results are structural and would not change across runs.
- **Supermemory's 86.1% is partly ingestion latency**, not a pure retrieval verdict. We warmed each hosted scope before querying and some still were not searchable in time; a longer warm window would likely raise it.
- **Write-time poisoning is mitigated, not solved.** The validator catches contradictions and low-confidence writes; hashing proves post-hoc immutability. Neither proves a value was true when written.
- **Retrieval was scoped per fact**, measuring cross-model transfer and gating, not precision when hundreds of facts compete—the regime where embedder and extraction quality matter most.

8 Related work

Hosted agent-memory platforms (mem0, Supermemory) and do-it-yourself retrieval over a vector store address the recall problem directly and, in our measurements, do so well. Provider-native memory (e.g., conversation or thread state) is scoped to a single vendor and therefore does not survive a cross-vendor routing hop, which is the failure mode of Section 2. Our contribution is not a new retrieval method—on recall we are at parity with these systems—but the measurement that recall is saturated across layers while verifiable, tamper-evident, model-attributed provenance is not, and that this provenance can be produced inline in the routing path at negligible latency cost.

9 Conclusion

Behind a model router, a memoryless agent fabricates most of the time, invisibly. Any competent memory layer fixes recall, so recall is the wrong axis on which to choose one. The axis that separates them is accountability: whether the system can produce a verifiable record of what each answer relied on and which model produced it. In a six-condition study, that record was present for 144 of 144 answers under SenseLab and absent under the alternatives, at a latency indistinguishable from adding no memory at all. Abstention on unanswerable questions remains a tunable tradeoff we report honestly rather than oversell.

A Reproducibility

Using SenseLab. SenseLab runs as a hosted service. Point the OpenAI SDK at it and add two headers to scope memory to an agent and an entity; your model gateway's routing, pricing, and failover are unchanged, and the layer fails open—if it errors, the request still reaches the router. Sign up for a key at `https://amfs.sense-lab.ai`.

```
from openai import OpenAI

client = OpenAI(
    base_url="https://amfs.sense-lab.ai/v1",   # was https://openrouter.ai/api/v1
    api_key=OPENROUTER_KEY,
    default_headers={"X-AMFS-Agent": "support", "X-AMFS-Entity": "acme/customer-42"},
)
```

Reproducing the measurements. The evaluation harness and the raw outputs behind every number in this paper are provided: `pro_benchmark.py` (six-arm recall, abstention, overhead), `gate_sweep.py` (the deterministic gate tradeoff), and `make_figures.py` (the figures), with results in `pro_results.json`, `gate_sweep.json`, and `pro_audit.jsonl` (144 sealed traces). Set `OPENROUTER_API_KEY`, `MEMO_API_KEY`, and `SUPERMEMORY_API_KEY`, then run:

```
python gate_sweep.py                                                    # -> gate_sweep.json
PRO_TRIALS=6 PRO_MISS_TRIALS=6 PRO_SEM_FLOOR=0.55 python pro_benchmark.py   # -> pro_results.json
```

References

[1] OpenRouter. *Unified API for LLMs — Documentation*. `https://openrouter.ai/docs`.

[2] mem0ai. *mem0: the memory layer for AI agents*. `https://github.com/mem0ai/mem0`.

[3] Supermemory. *Memory API for AI applications*. `https://supermemory.ai`.

[4] SenseLab. *SenseLab — the agent memory platform*. `https://amfs.sense-lab.ai`.

[5] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *J. Amer. Statist. Assoc.*, 22(158):209–212, 1927.